

Practical no 3

Bigram language Model with Smoothing and Perplexity

◆ 1. Language Models

A **language model** assigns probabilities to sequences of words.

- It helps us estimate how likely a sentence is.
- Applications: Speech recognition, Machine translation, Text generation, Chatbots.

◆ 2. N-gram Language Models

An **N-gram model** assumes that the probability of a word depends only on the previous $N-1$ words.

- **Unigram model (N=1):**

$$P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i)$$

- **Bigram model (N=2):**

Each word depends only on the previous one.

$$P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$$

Here:

- $P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$

3. Smoothing in Bigram Models

Problem: Some word pairs may never occur in training data, giving zero probability, which makes the whole sentence probability = 0.

To fix this, we use smoothing.

(a) Laplace (Add-1) Smoothing

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + 1}{\text{Count}(w_{i-1}) + V}$$

- Adds 1 to every possible bigram count.
- V = vocabulary size.

(b) Add-k Smoothing (generalization)

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i) + k}{\text{Count}(w_{i-1}) + kV}$$

- When $k = 1$, it becomes Laplace smoothing.
- Smaller k (e.g., 0.5) avoids over-penalizing frequent words.

(c) Stupid Backoff

If bigram unseen → fall back to unigram probability:

$$Score(w_i|w_{i-1}) = \begin{cases} \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})} & \text{if seen} \\ \alpha \cdot \frac{\text{Count}(w_i)}{\text{Total tokens}} & \text{if unseen} \end{cases}$$

where α is a discount factor (commonly 0.4).

⚠ It's a *score*, not a true probability.

(d) Interpolation (Linear Combination)

Combines **unigram** and **bigram** probabilities:

$$P(w_i|w_{i-1}) = \lambda_1 P(w_i) + \lambda_2 P(w_i|w_{i-1})$$

where $\lambda_1 + \lambda_2 = 1$.

◆ 4. Perplexity

Perplexity is a standard way to evaluate language models.

It measures **how well a model predicts a test set**.

- Definition:

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

- Equivalent using logs:

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i|w_{i-1}) \right)$$

Where:

- N = total number of words (including `<s>` and `</s>`).
- Lower perplexity = better model (more confident and accurate).

◆ 5. Example

Sentence: "I love machine learning"

Tokens: `<s>` I love machine learning `</s>`

Model calculates:

$$P(I | <s>) \cdot P(\text{love} | I) \cdot P(\text{machine} | \text{love}) \cdot P(\text{learning} | \text{machine}) \cdot P(</s> | \text{learning})$$

Then apply **smoothing** to avoid zero probabilities.

Finally, compute **perplexity**:

$$PP = \exp \left(-\frac{1}{6} \sum_{i=1}^6 \log P(w_i|w_{i-1}) \right)$$

Code

✓
0s



```
from collections import defaultdict, Counter
import math

class BigramModel:
    def __init__(self, corpus, k=1):
        self.k = k # for add-k smoothing
        self.vocab = set()
        self.unigram_counts = Counter()
        self.bigram_counts = defaultdict(Counter)
        self.total_tokens = 0
        self.corpus = corpus # save original corpus
        self._train(corpus)

    def _train(self, corpus):
        for sentence in corpus:
            tokens = ["<s>"] + sentence.split() + ["</s>"]
            self.total_tokens += len(tokens)
            self.vocab.update(tokens)
            for i in range(len(tokens) - 1):
                self.unigram_counts[tokens[i]] += 1
                self.bigram_counts[tokens[i]][tokens[i+1]] += 1
            self.unigram_counts[tokens[-1]] += 1 # For </s>
        self.V = len(self.vocab)
```

```
def laplace_prob(self, w1, w2):
    num = self.bigram_counts[w1][w2] + 1
    denom = self.unigram_counts[w1] + self.V
    return num / denom

def add_k_prob(self, w1, w2):
    num = self.bigram_counts[w1][w2] + self.k
    denom = self.unigram_counts[w1] + self.k * self.V
    return num / denom

def stupid_backoff_score(self, w1, w2, alpha=0.4):
    if self.bigram_counts[w1][w2] > 0:
        return self.bigram_counts[w1][w2] / self.unigram_counts[w1]
    elif self.unigram_counts[w2] > 0:
        return alpha * (self.unigram_counts[w2] / self.total_tokens)
    else:
        return alpha / self.total_tokens

def large_lang_interpolation(self, w1, w2, lambda1=0.25, lambda2=0.75):
    p1 = self.unigram_counts[w2] / self.total_tokens # unigram prob of w2
    p2 = self.bigram_counts[w1][w2] / self.unigram_counts[w1] if self.unigram_counts[w1] > 0 else 0
    return lambda1 * p1 + lambda2 * p2
```

```

def perplexity(self, sentences):
    """Compute perplexity for given sentences (list of strings)."""
    if isinstance(sentences, str): # allow single sentence input
        sentences = [sentences]

    log_prob = 0
    total_words = 0
    for sentence in sentences:
        tokens = ["<s>"] + sentence.split() + ["</s>"]
        total_words += len(tokens)
        for i in range(len(tokens)-1):
            w1, w2 = tokens[i], tokens[i+1]
            prob = self.laplace_prob(w1, w2) # use Laplace smoothing
            log_prob += math.log(prob + 1e-12)
    return math.exp(-log_prob / total_words)

def print_bigram_stats(self, w1, w2):
    print(f"Bigram: ({w1}, {w2})")
    print(f"Laplace P({w2}|{w1}) = {self.laplace_prob(w1, w2):.6f}")
    print(f"Add-k (k={self.k}) P({w2}|{w1}) = {self.add_k_prob(w1, w2):.6f}")
    print(f"Large Model Interpolation P({w2}|{w1}) = {self.large_lang_interpolation(w1,w2):.6f}")
    print(f"Stupid Backoff score({w2}|{w1}) = {self.stupid_backoff_score(w1, w2):.6f}")
    print("-" * 50)

```

```

# Example usage:
if __name__ == "__main__":
    corpus = [
        "i love machine learning",
        "i love deep learning",
        "machine learning is fun",
        "deep learning is powerful"
    ]

    model = BigramModel(corpus, k=0.5)

    queries = [
        ("i", "love"),
        ("love", "machine"),
        ("machine", "learning"),
        ("deep", "learning"),
        ("learning", "is"),
        ("is", "fun"),
        ("is", "amazing")
    ]

    for w1, w2 in queries:
        model.print_bigram_stats(w1, w2)

    # Compute perplexity for new unseen sentence
    new_sentence = "i love deep learning"
    print(f"\nPerplexity for '{new_sentence}': {model.perplexity(new_sentence):.6f}")

```

Output:

```
⇒ Bigram: (i, love)
Laplace P(love|i) = 0.250000
Add-k (k=0.5) P(love|i) = 0.357143
Large Model Interpolation P(love|i) = 0.770833
Stupid Backoff score(love|i) = 1.000000
-----

Bigram: (love, machine)
Laplace P(machine|love) = 0.166667
Add-k (k=0.5) P(machine|love) = 0.214286
Large Model Interpolation P(machine|love) = 0.395833
Stupid Backoff score(machine|love) = 0.500000
-----

Bigram: (machine, learning)
Laplace P(learning|machine) = 0.250000
Add-k (k=0.5) P(learning|machine) = 0.357143
Large Model Interpolation P(learning|machine) = 0.791667
Stupid Backoff score(learning|machine) = 1.000000
-----

Bigram: (deep, learning)
Laplace P(learning|deep) = 0.250000
Add-k (k=0.5) P(learning|deep) = 0.357143
Large Model Interpolation P(learning|deep) = 0.791667
Stupid Backoff score(learning|deep) = 1.000000
-----

Bigram: (learning, is)
Laplace P(is|learning) = 0.214286
Add-k (k=0.5) P(is|learning) = 0.277778
Large Model Interpolation P(is|learning) = 0.395833
Stupid Backoff score(is|learning) = 0.500000
-----

Bigram: (is, fun)
Laplace P(fun|is) = 0.166667
Add-k (k=0.5) P(fun|is) = 0.214286
Large Model Interpolation P(fun|is) = 0.385417
Stupid Backoff score(fun|is) = 0.500000
-----

Bigram: (is, amazing)
Laplace P(amazing|is) = 0.083333
Add-k (k=0.5) P(amazing|is) = 0.071429
Large Model Interpolation P(amazing|is) = 0.000000
Stupid Backoff score(amazing|is) = 0.016667
-----

Perplexity for 'i love deep learning': 3.575862
```

